Lerobot-Task

Motivation -> Lerobot currently training in CVAE uses just the joints and actions and stuff to get the z variable, but we wanted to test out what happens if we add images in it as well so the model can actually 'see' the environment while it learns. To test this, I modified the code and used the 'Place puck on shelf' task in MetaWorld. This task is a good test because the puck starts in a different spot every time. I trained two versions of the model—the standard one and my modified one—and compared them to see if adding images helps the robot adjust to the changing positions better.

Hypothesis -> We thought that adding more information while training the model must lower the losses of the modified act and it would perform much better under noisy instances and be more robust.

Modifications -> We were majorly concerned with making changes in 3 files mainly to perform and test this task. First to perform this task we had to make changes in the configuration_act.py file to add a bool for using images in the vae encoder.

// configuration_act.py

```
121     n_decoder_layers: int = 1
122     # VAE.
123     use_vae: bool = True
124     latent_dim: int = 32
125     n_vae_encoder_layers: int = 4
126     use_images_in_vae_encoder: bool = True  # Modified ACT: feed images to VAE encoder
127
128     # Inference.
129     # Note: the value used in ACT when temporal ensembling is enabled is 0.01.
130     temporal_ensemble_coeff: float | None = None
131
```

// modeling_act.py

```
323         # Backbone for image feature extraction.
324         if self.config.image_features:
325             backbone_model = getattr(torchvision.models, config.vision_backbone)(
326                 replace_stride_with_dilation=[False, False, config.replace_final_stride_with_dilation],
327                 weights=config.pretrained_backbone_weights,
328                 norm_layer=FrozenBatchNorm2d,
329             )
330             # Note: The assumption here is that we are using a ResNet model (and hence layer4 is the final
331             # feature map).
332             # Note: The forward method of this returns a dict: {"feature_map": output}.
333             self.backbone = IntermediateLayerGetter(backbone_model, return_layers={"layer4": "feature_map"})
334
335             # Project global image feature to model dim for VAE encoder input
336             if self.config.use_vae and self.config.use_images_in_vae_encoder:
337                 self.vae_encoder_img_input_proj = nn.Linear(
338                     backbone_model.fc.in_features, config.dim_model
339                 )
340
341         # Transformer (acts as VAE decoder when training with the variational objective).
342         self.encoder = ACTEncoder(config)
343         self.decoder = ACTDecoder(config)
344
345         # Transformer encoder input projections. The tokens will be structured like
346         # [latent, (robot_state), (env_state), (image_feature_map_pixels)].
347         if self.config.robot_state_feature:
348             self.encoder_robot_state_input_proj = nn.Linear(
```

```
        batch_size = batch[OBS_IMAGES][0].shape[0] if OBS_IMAGES in batch else batch[OBS_ENV_STATE].shape[0]

        # 1. Extract Image Features (Compute ONCE to reuse in both VAE and Transformer)
        # We store the raw feature maps in a list.
        all_cam_features = []
        if self.config.image_features:
            for img in batch[OBS_IMAGES]:
                all_cam_features.append(self.backbone(img)["feature_map"])

        # Prepare the latent for input to the transformer encoder.
        if self.config.use_vae and ACTION in batch and self.training:
            # Prepare the input to the VAE encoder: [cls, (robot_state), (image_summary), *action_sequence].
            cls_embed = einops.repeat(
                self.vae_encoder_cls_embed.weight, "1 d -> b 1 d", b=batch_size
            )
            vae_encoder_input = [cls_embed]

            # A. Add Robot State
            if self.config.robot_state_feature:
```

Forces images into the vae_encoder →

```
            vae_encoder_input.append(robot_state_embed)

            # B. Add Image Features (NEW logic for use_images_in_vae_encoder)
            if self.config.use_images_in_vae_encoder and self.config.image_features:
                # 1. Global Average Pool per camera: (B, C, H, W) -> (B, C)
                img_summaries = [f.mean(dim=[2, 3]) for f in all_cam_features]
                # 2. Average across all cameras: List[(B, C)] -> (B, C)
                global_img_feat = torch.stack(img_summaries).mean(0)
                # 3. Project and reshape: (B, C) -> (B, 1, D)
                img_token = self.vae_encoder_img_input_proj(global_img_feat).unsqueeze(1)
                vae_encoder_input.append(img_token)

            # C. Add Actions
            action_embed = self.vae_encoder_action_input_proj(batch[ACTION])
            vae_encoder_input.append(action_embed)

            # Concatenate all tokens
            vae_encoder_input = torch.cat(vae_encoder_input, axis=1)

            # Prepare fixed positional embedding.
            pos_embed = self.vae_encoder_pos_enc.clone().detach()
```

//[metaworld.py]

To change the position of the object we had to change a line

```
211             observation (Dict[str, Any]): The initial formatted observation.
212             info (Dict[str, Any]): Additional info about the reset state.
213         """
214         super().reset(seed=seed)
215         import random
216         # --- CORRECTED "LITTLE BIT OF NOISE" BLOCK --- <---- fixed approach
217         # 1. Always use Task 0 (the one you trained on).
218         #    This ensures the shelf/goal are in the general location the model learned.
219         task_idx = random.randint(0, len(self.mt1.train_tasks) - 1)
220         self._env.set_task(self.mt1.train_tasks[task_idx])
221 
222         # 2. The noise comes from here!
223         #    By passing the seed to env.reset(), MetaWorld will naturally apply
224         #    small random jitters to the object and gripper positions.
225         raw_obs, info = self._env.reset(seed=seed)
226         # ------------------------------------------- <---- very small noise only
227 
```

//record_dataset.py

```
#!/usr/bin/env python

"""
Script to generate a dataset for MetaWorld 'shelf-place-v3' task using the
expert policy.
Ensures only successful episodes are saved to the LeRobot dataset format.
"""

import os
import shutil
from pathlib import Path

import numpy as np
import torch

# LeRobot imports
from lerobot.datasets.lerobot_dataset import LeRobotDataset
from lerobot.envs.metaworld import MetaworldEnv

# --- Configuration ---
REPO_ID = "lerobot/shelf-place-v3"
TASK_NAME = "shelf-place-v3"
NUM_EPISODES = 50
FPS = 50
ROOT_DIR = Path("data")
MAX_EPISODE_STEPS = 500
```

```
OBSERVATION_WIDTH = 480
OBSERVATION_HEIGHT = 480

os.environ["SVT_LOG"] = "0"
os.environ["FFREPORT"] = "level=quiet"


class MetaworldEnvWithRawObs(MetaworldEnv):
    """Extended MetaworldEnv that tracks raw observations for expert policy."""

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self._raw_obs = None

    def reset(self, seed=None, **kwargs):
        """Reset and store raw observation."""
        # Get raw observation directly from inner env
        self._raw_obs, _ = self._env.reset(seed=seed)
        observation = self._format_raw_obs(self._raw_obs)
        return observation, {"is_success": False}

    def step(self, action):
        """Step and store raw observation."""
        if action.ndim != 1:
            raise ValueError(f"Expected action to be 1-D, got shape
{action.shape}")

        self._raw_obs, reward, done, truncated, info = self._env.step(action)

        is_success = bool(info.get("success", 0))
        terminated = done or is_success
        info.update({
            "task": self.task,
            "done": done,
            "is_success": is_success,
        })

        observation = self._format_raw_obs(self._raw_obs)

        if terminated:
            info["final_info"] = {
                "task": self.task,
                "done": bool(done),
                "is_success": bool(is_success),
```

```
            }

        return observation, reward, terminated, truncated, info


def create_dataset() -> LeRobotDataset:
    """Create a new LeRobotDataset with the appropriate features for
MetaWorld."""

    # Full dataset path including repo_id
    dataset_path = ROOT_DIR / REPO_ID

    # Remove existing dataset directory if it exists
    if dataset_path.exists():
        print(f"Removing existing dataset at {dataset_path}")
        shutil.rmtree(dataset_path)

    features = {
        "observation.images.image": {
            "dtype": "video",
            "shape": (OBSERVATION_HEIGHT, OBSERVATION_WIDTH, 3),
            "names": ["height", "width", "channel"],
        },
        "observation.state": {
            "dtype": "float32",
            "shape": (4,),
            "names": ["x", "y", "z", "gripper"],
        },
        "action": {
            "dtype": "float32",
            "shape": (4,),
            "names": ["dx", "dy", "dz", "gripper"],
        },
        "next.reward": {
            "dtype": "float32",
            "shape": (1,),
            "names": ["reward"],
        },
        "next.success": {
            "dtype": "bool",
            "shape": (1,),
            "names": ["success"],
        },
    }
```

```
    dataset = LeRobotDataset.create(
        repo_id=REPO_ID,
        fps=FPS,
        root=ROOT_DIR,
        features=features,
        robot_type="sawyer",
        use_videos=True,
        image_writer_processes=0,
        image_writer_threads=4,
    )

    return dataset


def record_dataset():
    """Main function to record expert demonstrations for the shelf-place-v3
task."""
    global NUM_EPISODES

    print(f"🔧 Initializing MetaWorld environment for task: {TASK_NAME}")
    env = MetaworldEnvWithRawObs(
        task=TASK_NAME,
        camera_name="corner2",
        obs_type="pixels_agent_pos",
        render_mode="rgb_array",
        observation_width=OBSERVATION_WIDTH,
        observation_height=OBSERVATION_HEIGHT,
    )

    expert_policy = env.expert_policy
    print(f"✅ Expert policy loaded: {type(expert_policy).__name__}")

    print(f"📁 Creating dataset: {REPO_ID}")
    dataset = create_dataset()

    print(f"\n🚀 Starting recording for task: {TASK_NAME}")
    print(f"🎯 Target: {NUM_EPISODES} SUCCESSFUL episodes (failures will be
discarded)")
    print("-" * 60)

    success_count = 0
    total_attempts = 0
```

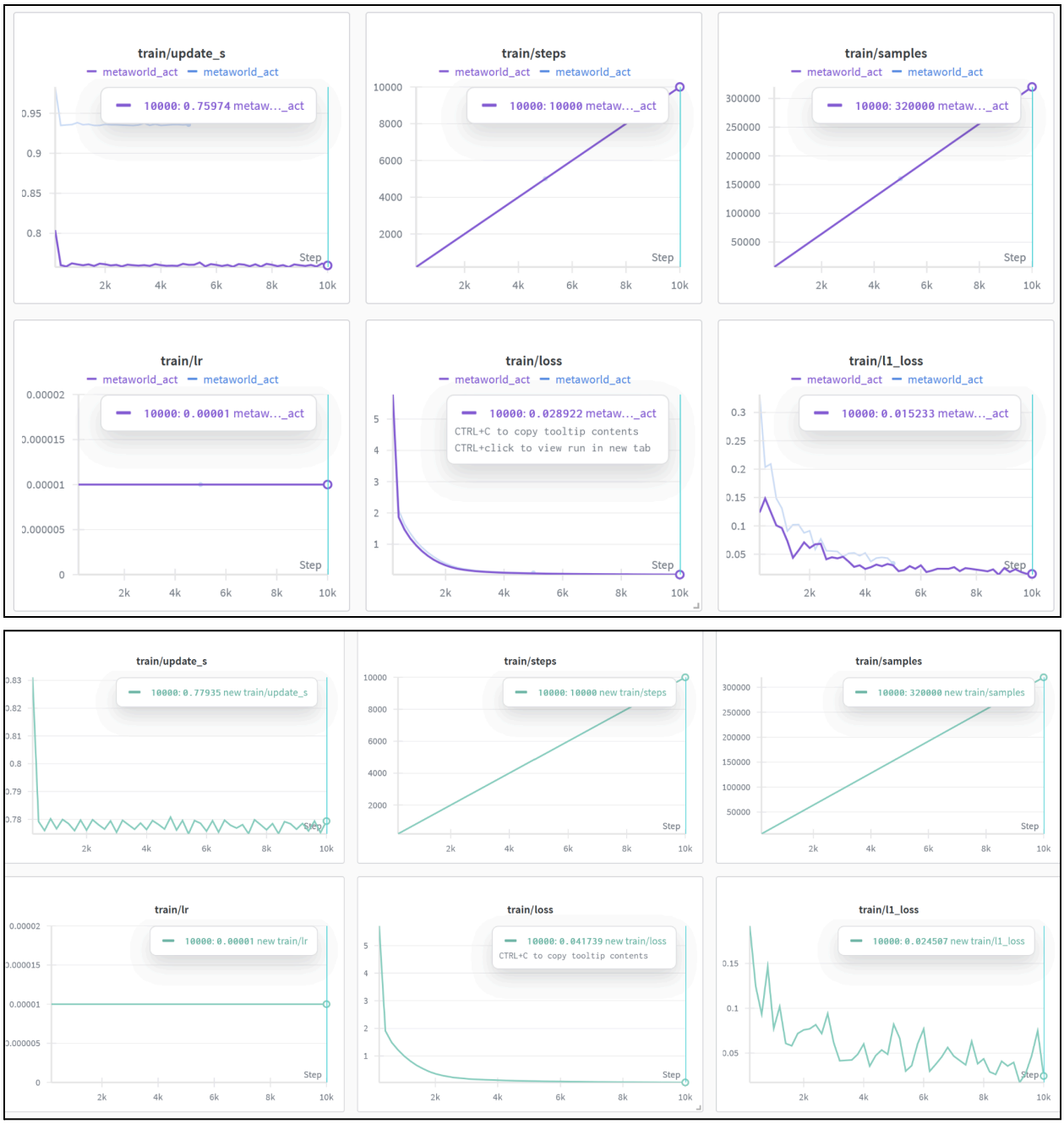
```
    try:
        while success_count < NUM_EPISODES:
            total_attempts += 1

            obs, info = env.reset()

            done = False
            step_count = 0
            episode_success = False

            dataset.episode_buffer = dataset.create_episode_buffer()

            while not done and step_count < MAX_EPISODE_STEPS:
                image = obs["pixels"]
                agent_pos = obs["agent_pos"]

                # Use the stored raw observation for expert policy
                action = expert_policy.get_action(env._raw_obs)
                action = np.clip(action, -1.0, 1.0).astype(np.float32)

                # Step the environment first to get reward/success
                next_obs, reward, terminated, truncated, step_info =
env.step(action)
                done = terminated or truncated

                # Create frame dict with current observation and next step info
                frame = {
                    "observation.images.image": image,
                    "observation.state": agent_pos.astype(np.float32),
                    "action": action,
                    "next.reward": np.array([reward], dtype=np.float32),
                    "next.success": np.array([step_info.get("is_success",
False)]),
                    "task": TASK_NAME,
                }

                if step_info.get("is_success", False):
                    episode_success = True

                dataset.add_frame(frame)

                obs = next_obs
                step_count += 1
```

```
            if episode_success:
                success_count += 1
                dataset.save_episode()
                print(f"✅ Episode {success_count}/{NUM_EPISODES} saved (Steps:
{step_count}, Attempts: {total_attempts})")
            else:
                dataset.episode_buffer = None
                print(f"❌ Attempt {total_attempts} failed after {step_count}
steps. Discarding...")

    except KeyboardInterrupt:
        print("\n⚠️ Recording interrupted by user")
    finally:
        print("\n" + "=" * 60)
        print("💾 Finalizing dataset...")
        dataset.finalize()
        env.close()

        print(f"\n📊 Dataset Summary:")
        print(f"   - Successful episodes: {success_count}")
        print(f"   - Total attempts: {total_attempts}")
        if total_attempts > 0:
            print(f"   - Success rate: {100 * success_count /
total_attempts:.1f}%")
        print(f"   - Saved to: {ROOT_DIR / REPO_ID}")
        print("\n✅ Done!")


def verify_dataset():
    """Utility function to verify the recorded dataset."""
    print("\n🔍 Verifying dataset...")

    try:
        dataset = LeRobotDataset(
            repo_id=REPO_ID,
            root=ROOT_DIR,
        )

        print(f"   - Total episodes: {dataset.num_episodes}")
        print(f"   - Total frames: {dataset.num_frames}")
        print(f"   - Features: {list(dataset.features.keys())}")
        print(f"   - FPS: {dataset.fps}")

        if len(dataset) > 0:
```

```
            sample = dataset[0]
            print(f"\n   Sample frame keys: {list(sample.keys())}")
            for key, value in sample.items():
                if isinstance(value, torch.Tensor):
                    print(f"   - {key}: shape={value.shape},
dtype={value.dtype}")
                else:
                    print(f"   - {key}: {type(value).__name__}")

        print("\n✅ Dataset verification passed!")
        return True

    except Exception as e:
        print(f"\n❌ Dataset verification failed: {e}")
        return False


if __name__ == "__main__":
    import argparse

    parser = argparse.ArgumentParser(description="Generate MetaWorld
shelf-place-v3 expert dataset")
    parser.add_argument("--verify-only", action="store_true", help="Only verify
existing dataset")
    parser.add_argument("--num-episodes", type=int, default=NUM_EPISODES,
help="Number of episodes to record")
    args = parser.parse_args()

    if args.verify_only:
        verify_dataset()
    else:
        NUM_EPISODES = args.num_episodes
        record_dataset()
        verify_dataset()
```

Work Proof-> lerobot → modified act , lerobot → normal act
Another copy of Untitled6.ipynb -> modified act, Untitled0.ipynb → eval of normal act.

Results → the loses curves after the training looked something like this for 1.) normal act and 2.) modified act, why such a big difference came during training of both? I have no answer, I am still clueless regarding it.



The Metrics → Evaluation was conducted using the LeRobot `bot_eval` pipeline with the following parameters: **Total Episodes:** 10 evaluation episodes per model, **Batching:** 5 batches **Inference Mode:** Temporal Ensembling enabled; VAE Encoder disabled (Latent z set to zero vector).

Metric	Baseline ACT (Standard)	Vision-Augmented ACT (Modified)
Success Rate	**90.0%**	60.0%
Avg. Sum Reward	**133.61**	92.45
Avg. Max Reward	**5.74**	4.12
Avg. Episode Length	**110 steps**	285 steps

Figure 4 respectively. The CVAE encoder only serves to train the CVAE decoder (the policy) and is discarded at test time. Specifically, the CVAE encoder predicts the mean and variance of the style variable z's distribution, which is parameterized as a diagonal Gaussian, given the current observation and action sequence as inputs. For faster training in practice, we leave out the image observations and only condition on the proprioceptive observation and the action sequence. The CVAE decoder, i.e. the policy, conditions on both z and the current observations (images + joint positions) to predict the action sequence. At test time, we set z to be the mean of the prior distribution i.e. zero to deterministically decode. The whole model is trained to maximize the log-likelihood of demonstration action chunks, i.e. $\min_\theta -\sum_{s_t, a_{t:t+k} \in D} \log \pi_\theta(a_{t:t+k}|s_t)$, with the standard VAE